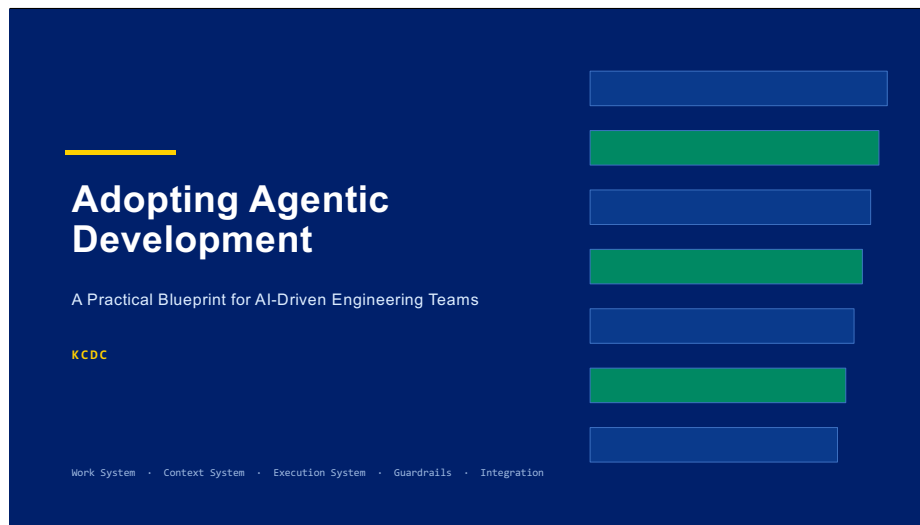




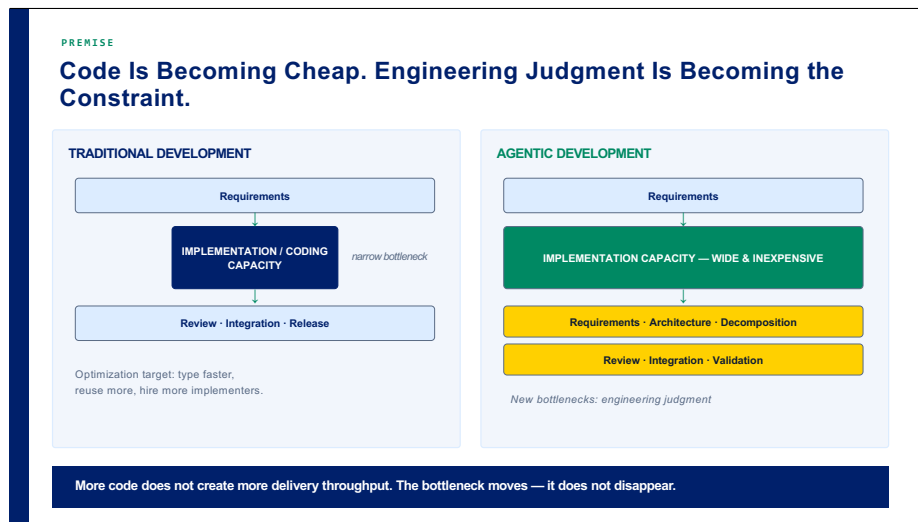
Open by explaining that this is not a talk about which coding agent is best. The session is about what happens after an organization moves past experimentation and starts allowing agents to perform meaningful engineering work.

The fundamental lesson from our experience is that adopting agents is not primarily a tooling problem — it becomes a systems-engineering problem.

We will focus on the architecture, work structures, guardrails, context systems, and operating practices required to make agentic development repeatable and reliable. Roughly 33 minutes of content, then a live demo, then questions.

Quick note the demo has changed a bit but I did not update the deck instead of a local inference we are going to use cloud inference and a different harness Personally at home I run an AMD Hellos based local infrance and a Invida DGX Spark with local models for company base actions we use github copilot

Transition: before we get into the presentation lets review the sponsors then we will move on to the economics.



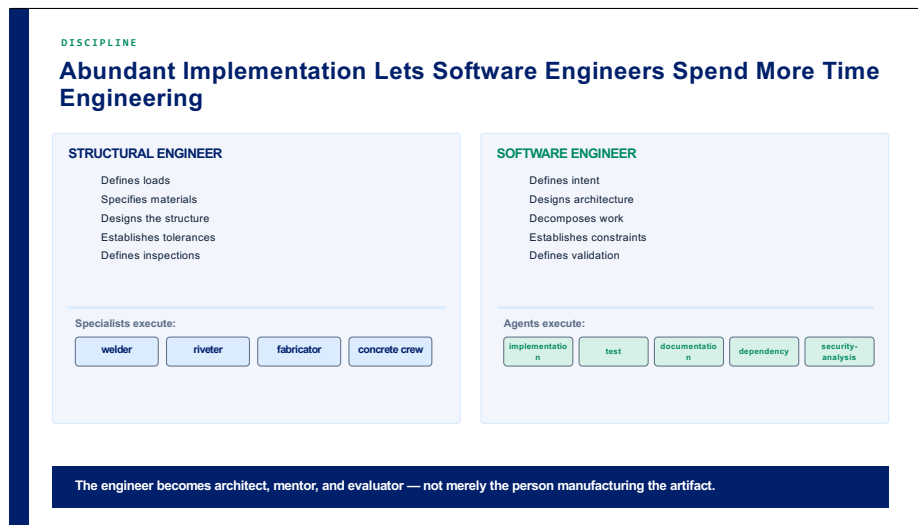
This is the central premise of the talk.

Historically most software optimization focused on increasing implementation throughput because writing and modifying code was expensive. Coding agents radically change that assumption.

Once implementation throughput increases, the constraints do not disappear — they move. In our organization the largest pressure moved toward PR review, work intake quality, feature decomposition, architecture decisions, product decisions, security and legal processes, documentation quality, and Support's ability to absorb change.

The purpose of an agentic engineering system is not to maximize generated code. It is to optimize the whole value-delivery system around this new economic reality.

Transition: this is not theory — here is where the evidence comes from.



This is one of the philosophical anchors of the talk. A structural engineer does not personally perform every weld or install every fastener in a bridge. The engineer designs the system and establishes the constraints within which specialists work.

Software has been unusual: engineers both designed the system and personally manufactured most of the artifact — largely because implementation was expensive. Software behaved more like a craft than a mature engineering discipline.

As implementation becomes cheap, we can spend more time on architecture, constraints, decomposition, review, and judgment. That is not the disappearance of engineering; it is the opportunity for software to behave like one.

Transition: here is the reference architecture for the rest of the talk.



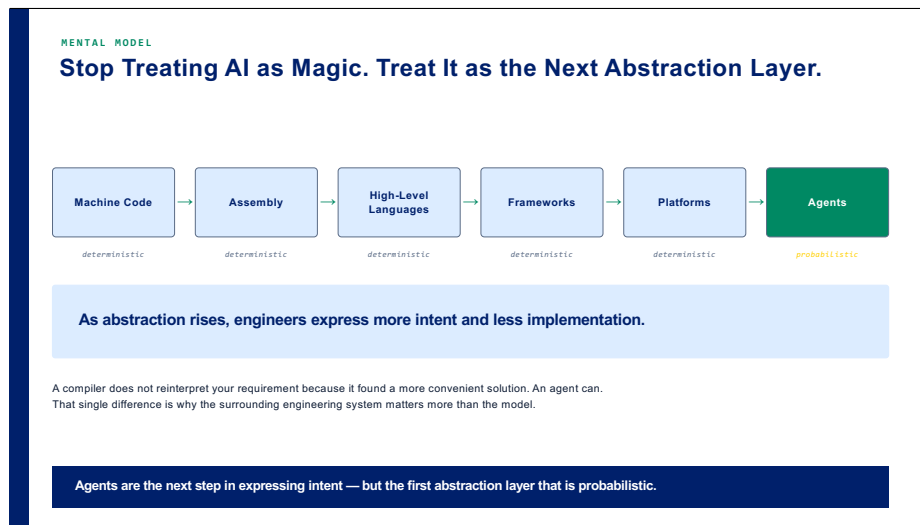
Establish context without presenting the organization as fully autonomous.

There are more than 600 engineers globally, but this talk focuses primarily on experience across more than 100 engineers in infrastructure-related products. We have been actively working through this transition for roughly 18 months.

Most teams are still progressing through Crawl and Walk. Some teams and specific workflows operate at Run, with selected workflows approaching Sprint.

This matters because the talk is not a claim that every team should immediately adopt highly autonomous agents. The progression matters.

Transition: to make the progression work, you need the right mental model for what an agent actually is.

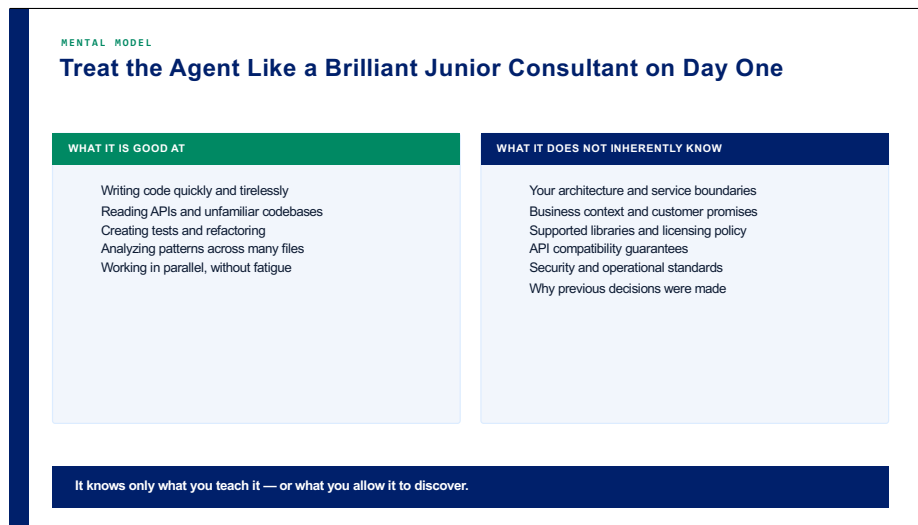


Software engineering has continually moved upward through abstraction layers. Each generation lets us specify more of what we want and less of exactly how the machine should perform it.

Agents are another step in this evolution: instead of specifying each implementation step, engineers specify intent, constraints, and outcomes.

The difference is that this abstraction is probabilistic. A compiler does not decide to reinterpret your requirement. An agent can. That difference drives everything else in this talk.

Transition: so what exactly is this new abstraction on day one? It is a very fast stranger.

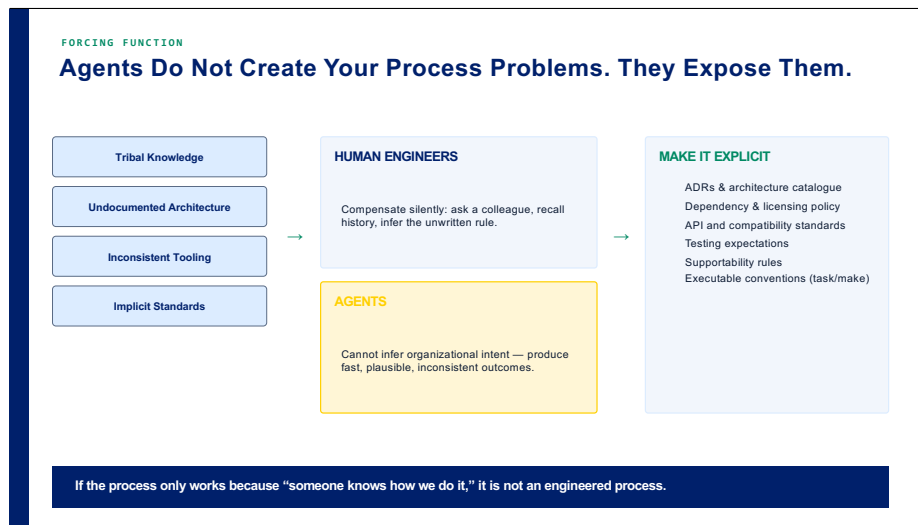


The useful mental model is not 'AI expert.' Imagine a very capable junior engineer arriving from a consulting firm: fast, technically competent, willing to work constantly — and with zero tribal knowledge.

They do not know why a service boundary exists, why one dependency is supported and another prohibited, which API compatibility promises customers depend on, or what your operational requirements are.

Humans fill these gaps through informal communication and experience. Agents force us to make that knowledge explicit.

Transition: which means agents put a spotlight on the process debt we were already carrying.

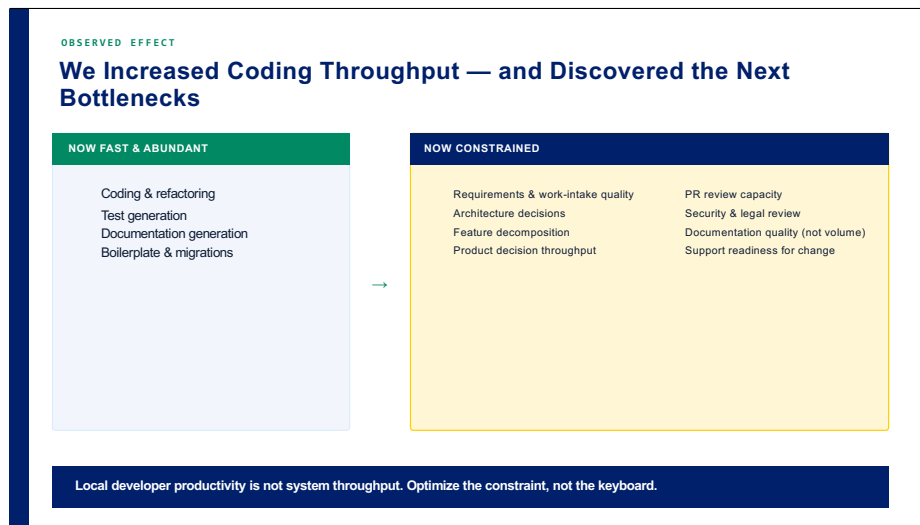


Make a brief parallel to the EOS operating-system philosophy on process discipline and tribal knowledge.

Organizations frequently believe they have a defined engineering process when much of it actually lives in conversations, personal experience, and historical convention. Humans are extremely good at compensating for this. Agents are much less forgiving.

That makes ADRs, dependency policy, API standards, testing expectations, supportability rules, and execution conventions far more important than before. Agentic development is a forcing function for process maturity.

Transition: when we actually removed the implementation constraint, here is what we hit next.

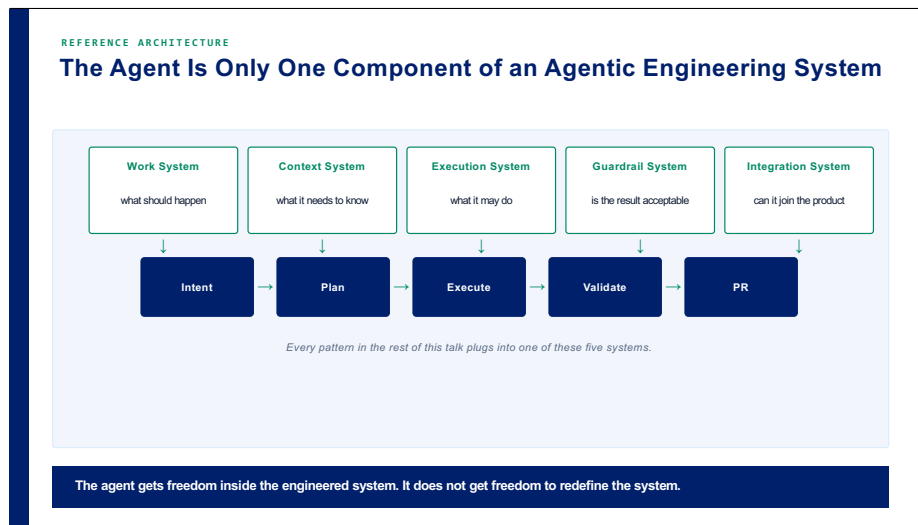


Explain the actual organizational effect. PR review became one of our largest constraints. Work-intake quality became far more visible, because agents will rapidly implement bad requirements.

Documentation quantity increased significantly; quality did not automatically follow. Security and legal processes are still adapting to increased delivery velocity. Support feels it too — more features and changes arrive faster.

The lesson: local developer productivity is not the same thing as system throughput.

Transition: this shift is actually an invitation — to do more real engineering.

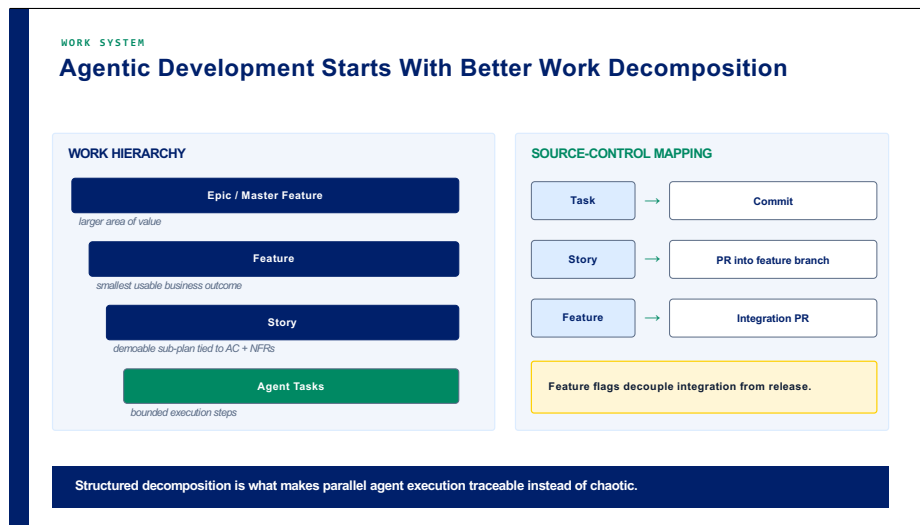


Introduce the reference architecture for the remainder of the presentation. The agent is not the engineering system.

The work system defines what should happen. The context system tells the agent what it needs to know. The execution environment determines what it is allowed to do. Guardrails determine whether the result is acceptable. Integration determines whether completed work can safely become part of the product.

The rest of the talk breaks this blueprint into implementable patterns — starting with the work system.

Transition: engineer the work before you engineer the code.



Define the hierarchy carefully. An Epic or Master Feature represents a larger area of value. A Feature is the smallest independently usable business outcome. Stories are demoable pieces that map back to acceptance criteria and non-functional requirements. Each step inside the story plan becomes an explicit task or agent task.

Each task normally results in a commit. The story results in a PR against the feature branch. The feature integrates into an epic branch or the mainline depending on repository strategy.

This structure supports parallel agent execution without losing traceability. Feature flags matter because integration and release should not be unnecessarily coupled.

Transition: within that structure, what makes a single piece of work actually ready for an agent?

REUSABLE TEMPLATE

Before an Agent Starts, the Work Must Be Agent-Ready

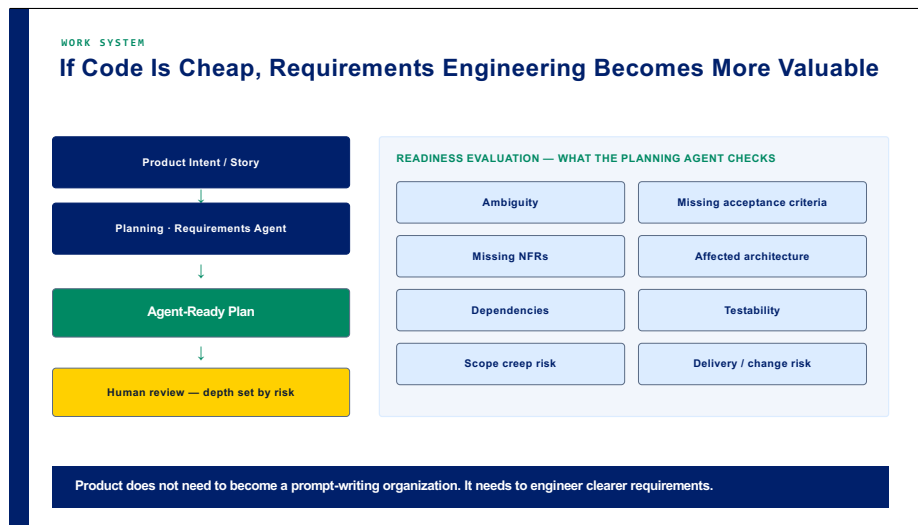
Intent What business or user outcome should exist	Scope Where change is allowed	Out of Scope Explicit boundaries agents must not expand
Acceptance Criteria Observable, testable success	Non-Functional Requirements Security, performance, compatibility, operability	Architecture Constraints Relevant ADRs, boundaries, patterns
Allowed Operations Which commands / tools may be executed	Validation How correctness will be demonstrated	Definition of Complete All requirements met, no unrelated behavior change, guardrails pass

A ticket existing does not mean it is ready for autonomous execution.

This slide must pass the screenshot test exceptionally well — tell the audience to photograph it.

Intent tells the agent what outcome should exist. Scope defines where change is allowed. Out of Scope is critical because agents frequently expand their interpretation. Acceptance Criteria defines observable success. NFRs cover security, performance, compatibility, operability, supportability. Architecture Constraints point at ADRs and boundaries. Allowed Operations define permitted execution. Validation specifies how correctness is demonstrated. Definition of Complete means all requirements satisfied without materially changing unrelated behavior, with all guardrails passing.

Transition: producing work in this shape at scale is itself a job — and agents can help.



A major organizational implication is that work preparation has to improve. Product should not be responsible for writing perfect AI prompts. Product should increasingly practice requirements engineering: outcomes, constraints, user value, acceptance expectations.

Agents can help evaluate and enrich that work before another agent implements it. A planning agent can flag missing AC, missing NFRs, unclear scope, architecture dependencies, and risks.

Not every plan requires the same human checkpoint — risk determines oversight.

Transition: notice I said 'another agent.' Planning and execution are different jobs.

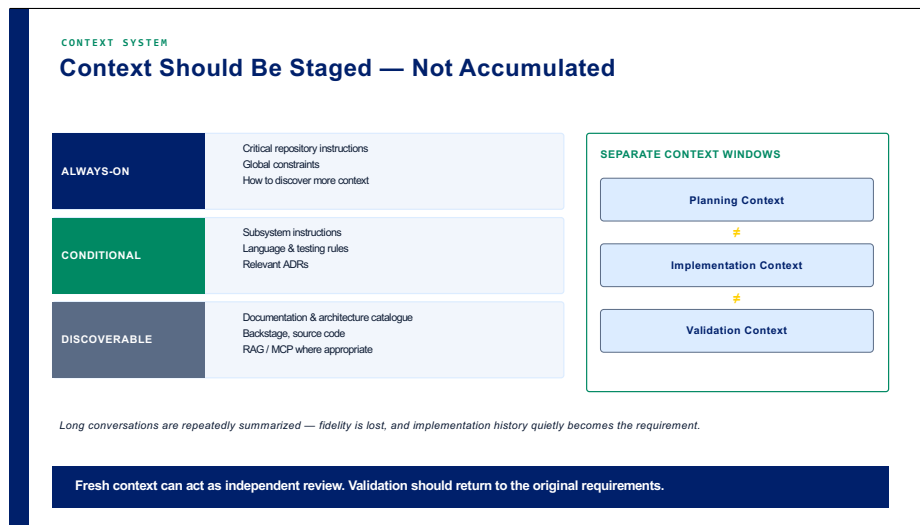


Planning is typically where expensive reasoning provides the most value. Implementation can often be done by smaller, local, or task-specific models. Other activities use different models again.

A model that plans well may not write the best human-facing documentation. A specialized coding model may be unnecessary for coverage analysis.

The important architecture is not 'choose the best AI model.' The architecture should let models and agents be replaced according to the work.

Transition: the other half of that separation is context — which is itself an architecture.

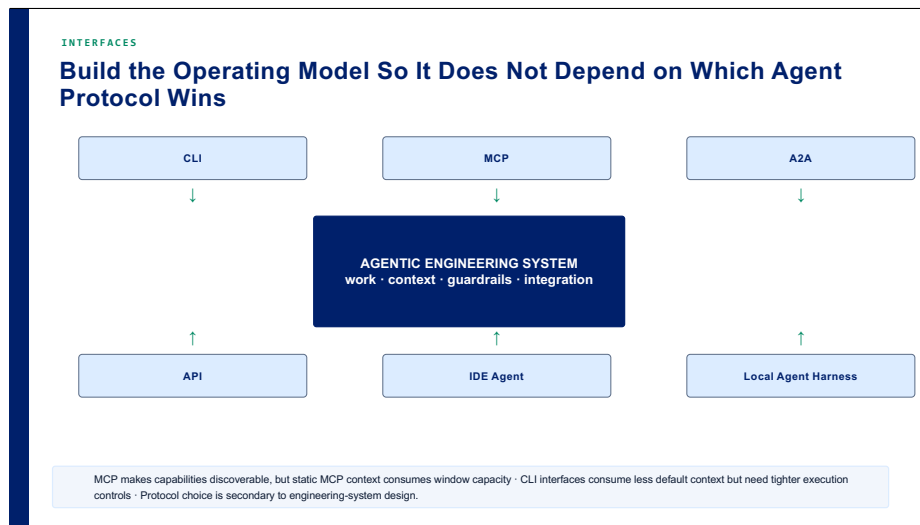


Large context windows do not eliminate context-management problems. Static context works, but not everything belongs in one giant instructions file.

Some knowledge must always be present. Some should load conditionally. Some should be discoverable only when needed.

Long conversations get repeatedly summarized and fidelity is lost — we have token-usage and agent-turn data showing the effect of long execution chains. Use fresh contexts for distinct phases so implementation history does not become the sole source of truth. Validation returns to the original requirements and plan.

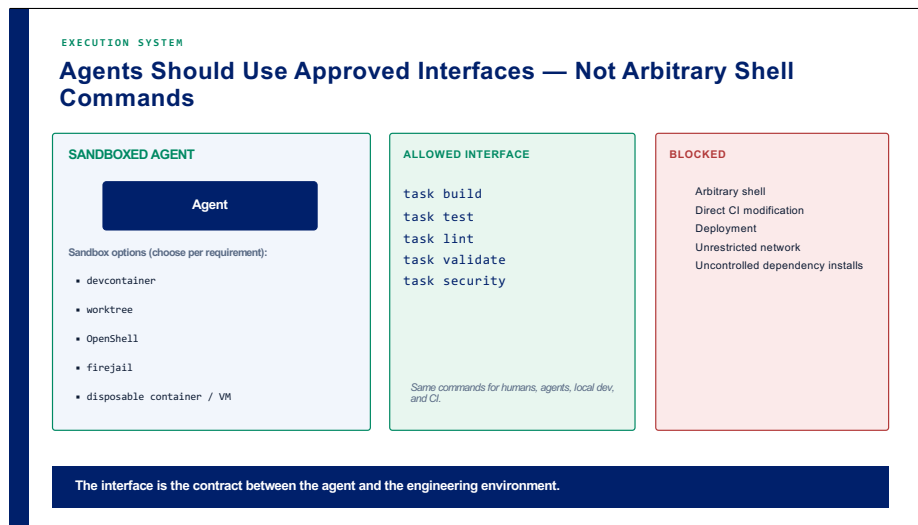
Transition: a quick word on protocols — because everything in this space is changing monthly.



Keep this brief. The ecosystem is changing quickly. MCP, A2A, CLI interfaces, IDE agents and others will keep evolving.

Teams should understand the tradeoffs — especially how a protocol affects context usage and execution authority. But the core operating model should survive a change of protocol or vendor. Do not build your organizational process around one tool's current implementation.

Transition: whatever the protocol, you must constrain how the agent is allowed to act.

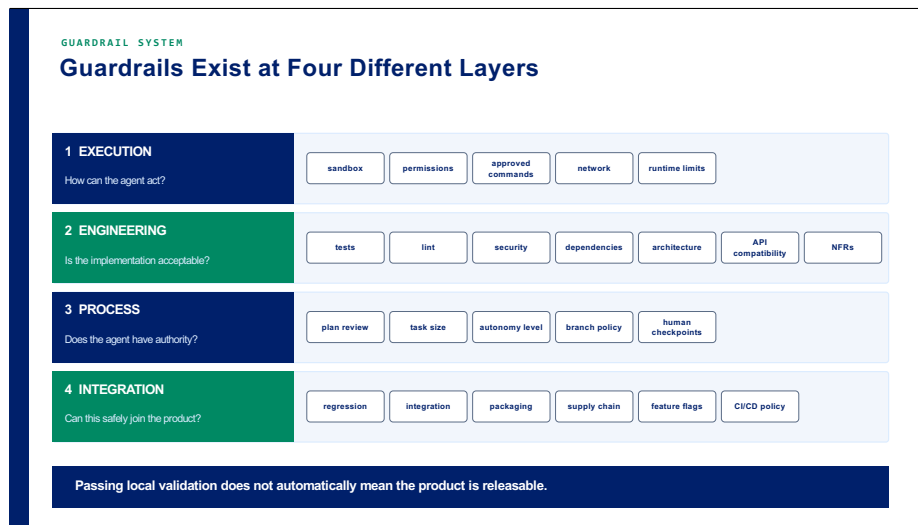


This is one of the biggest practical recommendations. Do not simply give an autonomous coding agent unrestricted shell access.

Agents should interact with the project through abstractions — Taskfile, Make, or equivalent project commands. That makes execution predictable and auditable, and the same command works for humans, agents, local development, and CI.

The implementation technology is flexible: devcontainer, worktree, OpenShell, firejail, or a disposable environment. The principle is bounded execution with explicit capability.

Transition: execution control is only one of four guardrail layers.



This classification helps teams avoid dumping every control into one bucket.

Execution controls capability. Engineering guardrails evaluate the produced implementation. Process guardrails determine authority. Integration guardrails evaluate the work in the context of the complete system.

The distinction between task completion and product release readiness is especially important — an agent's green build is not a release decision.

Transition: process guardrails raise the obvious question — how much autonomy?

AUTONOMY POLICY

Autonomy Should Be Risk-Based, Not Model-Based

Work Type	Plan Review	Agent Execution	Validation	Human PR Review
Dependency Update	Usually none	Autonomous	Strict automated checks	Yes
Documentation	Usually none	Autonomous	Link / style / content checks	Lightweight
Bug Fix	Sometimes	Autonomous	Tests + regression	Yes
Small Feature	Required	Autonomous after plan	Full engineering checks	Yes
Architecture Change	Mandatory	Highly bounded	Expanded validation	Senior review

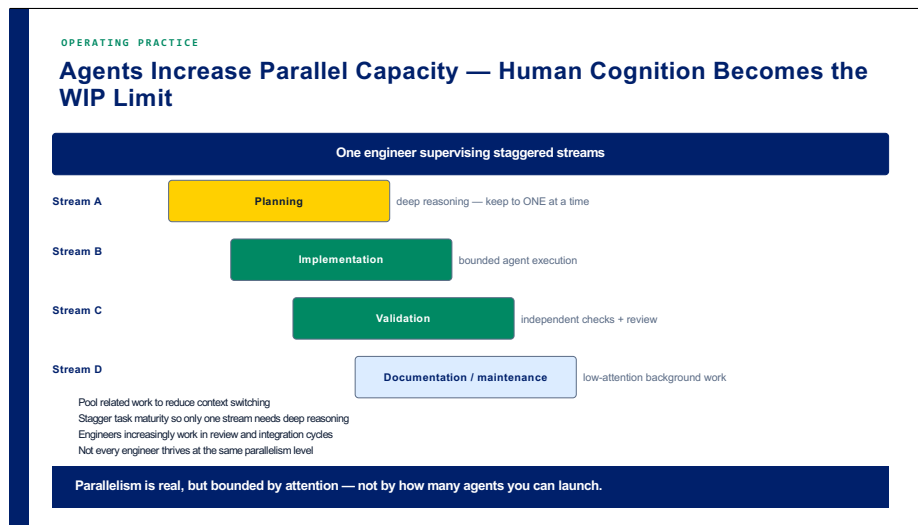
A dependency update and a new architecture should not have the same autonomy policy.

The common question 'should humans be in the loop?' is too vague. The answer depends on the work.

Routine, reversible, highly testable work can operate with significantly more autonomy. Work that changes architecture, contracts, security boundaries, or major behavior requires stronger human judgment.

The goal is not maximum autonomy. The goal is the appropriate amount of autonomy for the risk. Appendix E expands this into a fuller risk policy.

Transition: once autonomy is calibrated, you hit a very human limit — attention.

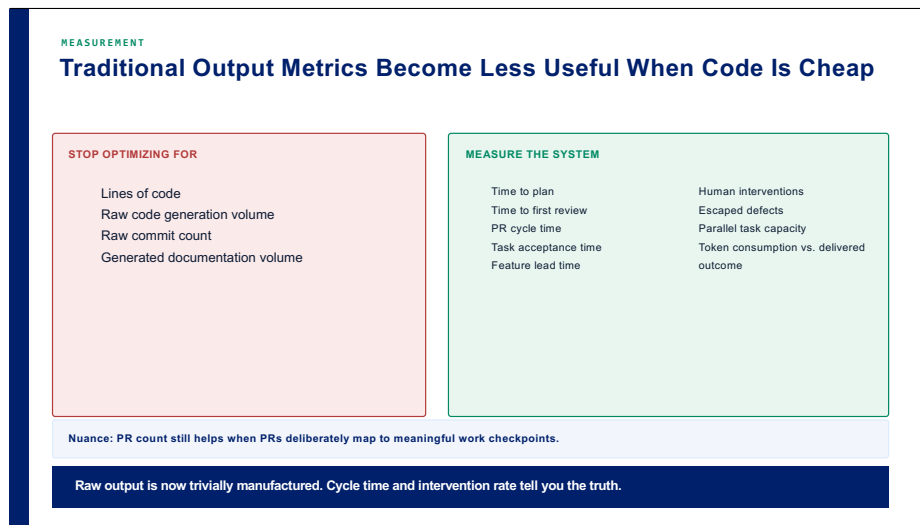


Parallelization is real but not unlimited. One person can supervise multiple tasks, but not every task should demand deep reasoning at the same time.

We generally keep only one substantial planning activity active. Other related work sits in implementation, validation, documentation, testing, or maintenance phases. Pool related tasks to cut context switching.

Not every engineer thrives under the same amount of parallel work — treat this as a per-person WIP limit, not a policy.

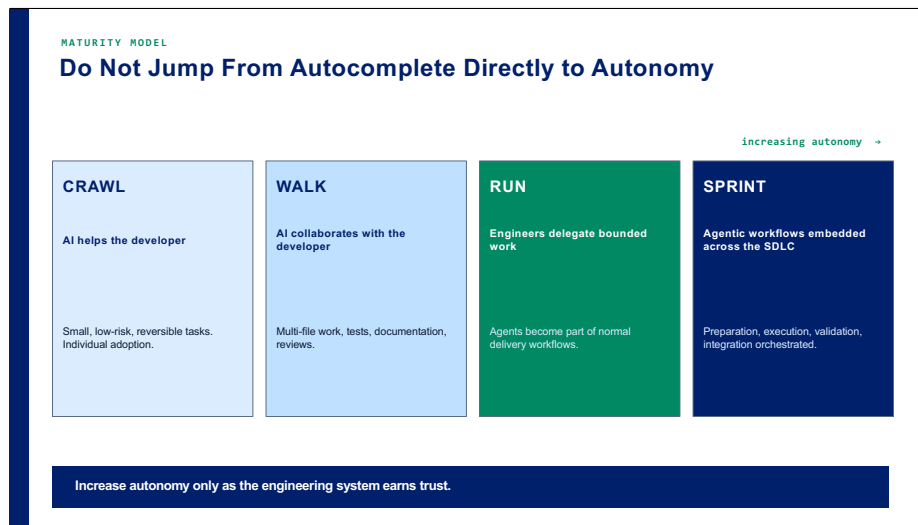
Transition: if the work looks different, the measurements have to change too.



Do not argue that every historical metric is meaningless. PR count still provides useful information when the branching and work model maps PRs to meaningful checkpoints.

But raw output is increasingly easy to manufacture. Cycle-time metrics become far more informative: measure time between stages, how often humans must intervene, whether defects escape, useful work relative to model usage, and how many parallel tasks an engineer can safely supervise.

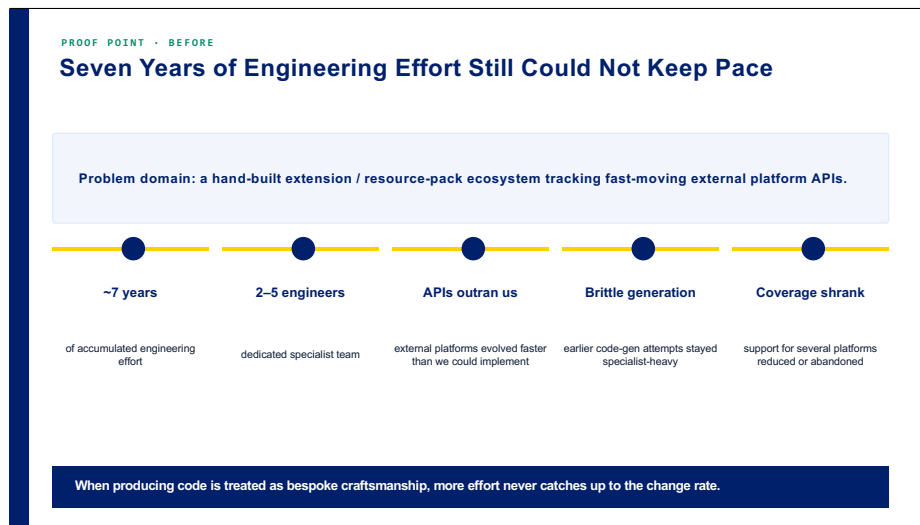
Transition: metrics tell you when you have earned the right to move up the maturity ladder.



Our maturity model starts with confidence and basic usage. Crawl is individual adoption. Walk makes AI part of the normal engineering workflow. Run begins meaningful delegation. Sprint represents deeper AI-native engineering practice across the SDLC.

Most organizations should not attempt to begin at Sprint. The architecture and engineering discipline must mature alongside autonomy — the guardrails, context, and work system have to exist before the autonomy is safe.

Transition: let me make this concrete with a real problem we could not solve for seven years.



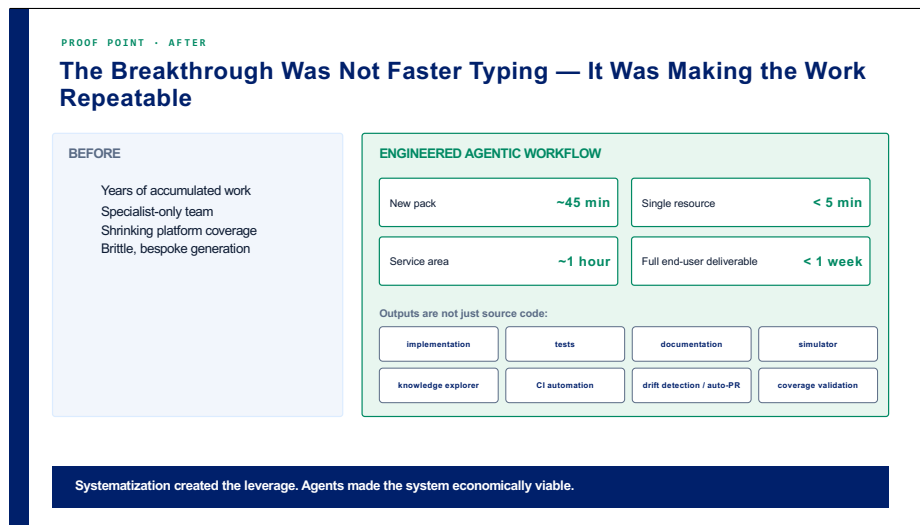
Introduce the real-world example while abstracting it enough to be broadly applicable.

The team maintained resource packs / extensions for rapidly changing infrastructure and cloud APIs. Across roughly seven years, a team of two to five engineers accumulated significant engineering effort — and the work remained expensive.

External APIs changed faster than humans could manually implement and maintain coverage. Earlier code-generation attempts reduced typing but stayed brittle and still demanded deep specialist knowledge. Coverage for some platforms was eventually reduced or abandoned.

This is exactly the type of problem where producing code was still bespoke craftsmanship.

Transition: what changed was not the typing speed.

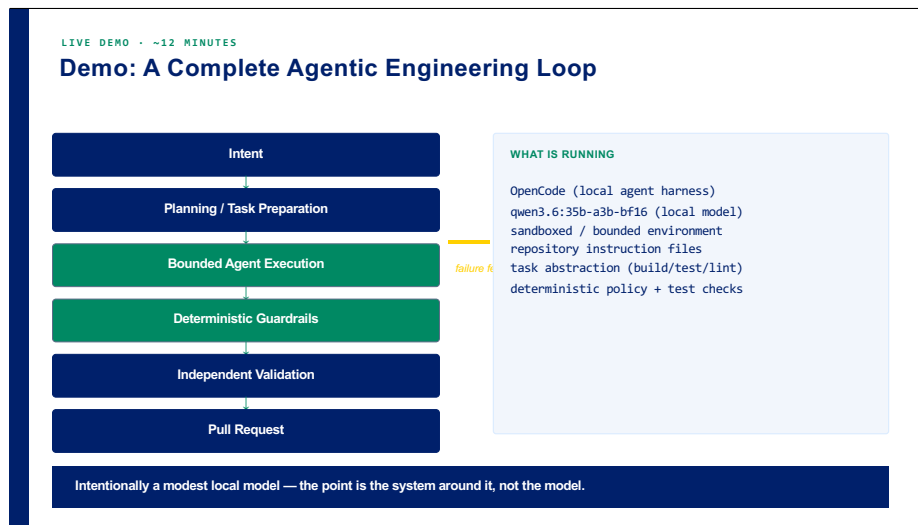


This is one of the strongest ROI examples. A new pack can now be created in about 45 minutes. Individual resources take under five minutes. Entire service areas take about an hour. An end-user-ready pack can be completed in less than a week.

More importantly, the result is not only source code: the engineered workflow produces tests, documentation, a simulator, knowledge-exploration capability, CI automation, drift detection with automated PRs, and coverage validation.

The lesson is not 'the AI writes code hundreds of times faster.' The lesson is that we converted a bespoke engineering activity into a repeatable engineered system.

Transition: let's watch a small version of that loop live.



Transition into the live demo. The purpose is not to show the smartest available model — we will intentionally use OpenCode with a local qwen3.6:35b-a3b-bf16 model.

The goal is to demonstrate that the surrounding engineering system matters more than model perfection. The agent receives bounded work, operates through approved interfaces, hits a deterministic guardrail, interprets that failure, corrects the work, validates the result, and raises a PR.

Stop the presentation here and perform the live demo. Budget 11–12 minutes; if you fall behind, skip the knowledge-explorer portion and go straight to the guardrail failure and recovery.

LIVE DEMO • ~12 MINUTES

Demo: A Complete Agentic Engineering Loop



Transition into the live demo. The purpose is not to show the smartest available model — we will intentionally use OpenCode with a local qwen3.6:35b-a3b-bf16 model.

The goal is to demonstrate that the surrounding engineering system matters more than model perfection. The agent receives bounded work, operates through approved interfaces, hits a deterministic guardrail, interprets that failure, corrects the work, validates the result, and raises a PR.

Stop the presentation here and perform the live demo. Budget 11–12 minutes; if you fall behind, skip the knowledge-explorer portion and go straight to the guardrail failure and recovery.

AFTER THE DEMO

A Good Engineering System Is Designed to Catch Imperfect Implementation



Mistakes are expected. What matters is that they occur inside a bounded environment and are caught by deterministic controls before they become integrated product behavior.

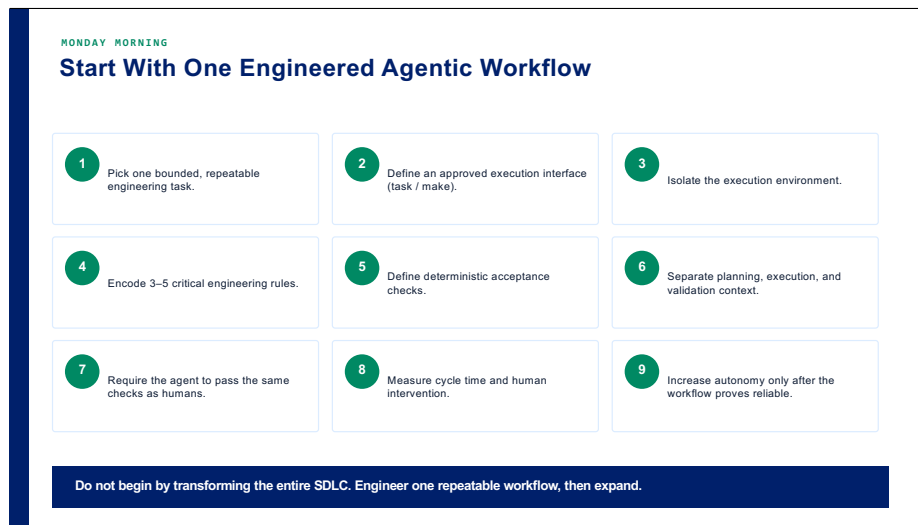
Do not design a system that requires the model to be right on the first try.

Use this immediately after the demo. The successful outcome is not that the model made no mistakes.

The system should assume agents will occasionally make imperfect choices. The important property is that those mistakes happen inside a bounded environment and are discovered by deterministic controls before they become integrated product behavior.

Engineering the feedback loop is more scalable than hunting for a perfect model.

Transition: so what do you actually do on Monday?



This is the practical call to action. Attendees should not go home and attempt to build a multi-agent autonomous engineering organization.

Choose one repeatable task — dependency updates, a defined extension pattern, small maintenance changes, or another bounded workflow. Make the process deterministic before making it autonomous. Measure the result. Improve the system. Then expand.

Transition: let me close by returning to the three ideas the whole talk rests on.

Agentic Development Is an Engineering-System Problem

- 1 Stop treating AI as magic. Treat it as the next abstraction layer.
- 2 Code is becoming cheap. Engineering judgment is becoming the constraint.
- 3 Probabilistic workers require deterministic engineering systems.

The goal is not autonomous code generation.
The goal is reliable business outcomes from abundant implementation capacity.

Questions - Appendix slides follow

Close by returning to the central argument. Agents do not eliminate the need for software engineering discipline — they increase its importance.

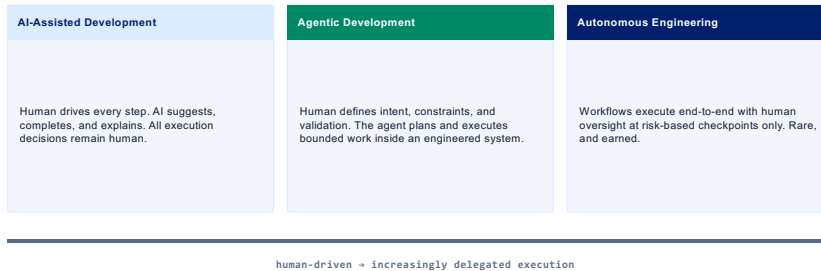
As implementation becomes inexpensive, the engineering organization has the opportunity to move more human effort toward architecture, requirements, constraints, design, review, and validation.

The organizations that benefit most from agentic development will not be the ones that buy the best coding model. They will be the ones that engineer the systems around those models.

Thank the audience and move to questions. Appendix slides cover definitions, an example agent-ready story, repository structure, guardrail pipeline, risk policy, and context anti-patterns.

DEFINITIONS

Appendix A — A Spectrum, Not Three Products



Use in Q&A when terminology gets muddled. The point is that these are positions on a delegation spectrum, not three different products. Most organizations sit between assisted and agentic, with a handful of workflows further along.

Appendix B — Example Agent-Ready Story

Intent	Users can filter the audit-log API by date range so support can investigate incidents without exporting full logs.
Scope	services/audit-api query layer, request validation, OpenAPI spec, unit + integration tests.
Out of Scope	Storage schema changes, UI, retention policy, authn/authz behavior, other endpoints.
Acceptance Criteria	from/to ISO-8601 params; inclusive range; 400 on invalid or inverted range; default unchanged when omitted; paging preserved.
NFRs	p95 < 300ms at 10M rows; no new dependency; backward compatible; structured logs on rejection.
Architecture Constraints	ADR-014 query-builder boundary; no raw SQL in handlers; repository interface unchanged.
Allowed Operations	task build, task test, task lint, task validate. No network, no dependency installs, no CI edits.
Validation	New unit tests + integration test against the seeded fixture; contract test vs. published OpenAPI; existing suite green.
Definition of Complete	All AC + NFRs met, no unrelated behavior change, all four guardrail layers pass, PR opened against the feature branch.

A concrete, generic instance of the Agent-Ready Work Template from slide 11. Note how much of the value is in Out of Scope and Allowed Operations — those two fields prevent most of the surprises we used to see.

REPOSITORY STRUCTURE

Appendix C — Architecture, Instructions, Tasks, and Policies Are First-Class Artifacts

```
/docs
  /adr          architecture decision records
  /architecture catalogue, boundaries, diagrams

/agent
  instructions.md always-on repository rules
  /conditional    subsystem & language guidance

/tasks
  Taskfile.yml   approved execution interface

/src
/tests
/policies        dependency, license, security
/.devcontainer   sandbox definition
```

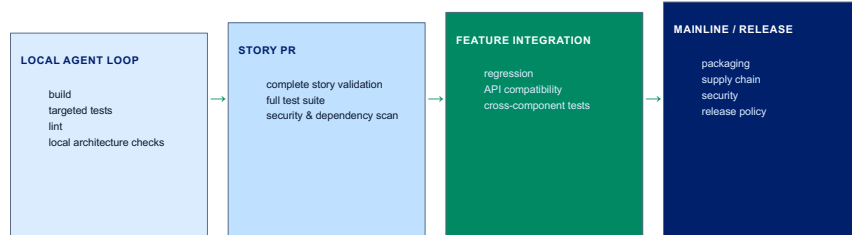
WHY IT MATTERS

Instructions live with the code they govern and version with it
Executable tasks give humans, agents, and CI one interface
Policies are enforceable, not aspirational
ADRs give agents the 'why' behind boundaries
Sandbox config makes bounded execution reproducible

Exact filenames are illustrative — the structure is the point.

Illustrative layout. The point is that architecture, instructions, executable tasks, policies, tests, and sandbox configuration are first-class repository artifacts — not wiki pages that drift.

Appendix D — Guardrails Broaden as Work Moves Toward Release



Scope of validation widens with blast radius — fast and narrow locally, broad and slow at release.

Use in Q&A. Emphasize that guardrails become progressively broader as work moves closer to integration — you cannot afford full regression on every agent turn, and you cannot afford to skip it at release.

Appendix E — Risk Dimensions Determine Autonomy, Review, and Validation Depth

RISK DIMENSIONS (SCORE EACH)

Reversibility — can it be rolled back cheaply?
Customer impact — who notices if it is wrong?
Architecture impact — does it move a boundary?
Security impact — authn, authz, secrets, data paths
API impact — published contract or compatibility
Data impact — migration, retention, correctness
Validation quality — how good are the existing checks?



WHAT THE SCORE SETS

Required planning review

none → peer → senior → architecture board

Permitted agent authority

autonomous → bounded → plan-gated → assisted only

Validation depth

targeted tests → full suite → regression + compatibility

Human review level

lightweight → standard → senior + second reviewer

Expansion of the slide 18 matrix. The practical advice: score the dimensions once per work type, not per ticket, and let the work type carry the policy. Re-score only when a work type's risk profile changes.

Appendix F — What Not to Do With Agent Context

✗ One enormous AGENTS.md that everything is appended to

✗ Putting every document into every prompt

✗ Using one conversation from planning through validation

✗ Relying on repeated summarization indefinitely

✗ Letting implementation history redefine the requirement

✗ Repeatedly fetching static knowledge that should be durable

Context is a budget. Spend it deliberately.

Q&A slide. Each anti-pattern maps to a failure we actually hit. The most damaging one is the third: a single long conversation means the agent's own implementation history becomes the requirement, and validation loses independence.